



The Viable Systems Model

The Viable Systems Model specifies a generalised model for agent-based systems. It breaks down self-adaptive software into six major systems:

System One (S1) - Operations: The dirty workers—one or more S1s perform the basic operations of the application.

System Two (S2) - Co-ordination: The foremen—S2s co-ordinate the activities of the S1s.

System Three - Control: The

management guys—resource planning and strategic planning.

System Three* - Audit: The inspector—monitoring the progress of the S1s and seeing whether all is going according to plan.

System Four (S4) - Intelligence: The navigator—surveys the environment and plans the way ahead.

System Five (S5) - Policy: The Big Boss—defines the purpose of the entire system, based on the 'world view' provided by S4.

see, is to pick up all three and then head to D. The long term consequences of the agent's actions are captured in the *utility function*. So the best solution for the agent will involve high immediate *and* long term rewards. In this way, Reinforcement Learning starts with an initial guess at the utility function, and evolves as it explores the system.

And finally, when our agent has explored the system enough and trained itself for the best utility function, it can be deployed into the real network.

What we've described here is a single controlling agent for our system. In a lot of applications, though, we would need to have many agents interacting with each other. The RL algorithm in such cases would need to be modified to incorporate complex theories such as the Game Theory and the Economic Market Theory.

All is not hunky-dory with RL, though. Like any human agent will tell you, you can't always get what you want. This holds good with the software agent as well. Everything cannot be optimised at once, so the software agent will make a few trade-offs, and you will need to tell the software agent how much importance you attach to resources and to results. You also never know what is going to happen—software may not be quick enough to respond to sudden changes in environment, and performance would suffer for a while.

Is It... Human?

Another way for agents to learn is the *Genetic Algorithm*, that's designed to achieve the same goals as Reinforcement Learning, but uses a different approach. Instead of learning the utility function to satisfy the best policy as time goes on, the Genetic Algorithm learns the best policy itself.

The reason Genetic Algorithms are called so is because of their close resemblance to biological evolution. The Genetic Algorithm creates a pool of possible policies and evaluates each one to see the rewards. By combining successful policies, a new pool of policies is created. This is akin to biological reproduction and the process of natural selection—by combining the best of the gene pool, a newer, stronger gene pool is created, and the rest just die. This process continues until the best possible policy is obtained—the software

To implement adaptive code, we need a Dynamic Language—one that allows software to be changed even at runtime

version of "Survival of the Fittest."

Deciding between an RL or Genetic Algorithm approach is difficult. Reinforcement Learning would work better in bigger, more complex systems, but Genetic Algorithms would be the preferred choice for a small system with limited scope - the fewer policies it has to sift through, the more efficient it becomes.

Decisions, Decisions

Thus far, we've talked about agents making decisions—but how do they "decide"? They are, after all, computer programs, and all they know is zero or one—true or false.

The answer is a *Probabilistic Network*, which executes an action based on the information presented to it by the surrounding environment. Decisions are made based on the probability of a favourable outcome from an action, given all that is happening in the surroundings. The term "Network" is misleading—probabilistic networks are a way of depicting probability problems, rather than being actual, physical networks.

We now need to talk about the *Bayesian* or *Belief* network, which makes decisions based on Bayes' Theorem, developed by the Rev Thomas Bayes. The gist of this theorem is that the strength of our belief in a hypothesis, given some additional evidence, depends on the probability of the hypothesis alone, as well as the probability of the evidence assuming the hypothesis itself is true.

By starting on an initial hypothesis, the software goes on to calculate all the possible Bayesian networks that would maximise the probability of the hypothesis being true, finally coming up with a structure that would give the maximum reward.

So we now have an agent using Bayesian networks to predict the utility function that will maximise its rewards. After it has made its initial prediction, it selects the best method to achieve this function and sets about executing this method. The agent will now monitor the rewards of this system and keeps modifying its idea of the utility function to reap greater rewards.

The Tools Of The Trade

To implement adaptive code, we need a Dynamic Language—one that allows software to be changed even at runtime. Languages like C demand that the programmer have a clear idea of the structure that the program is going to take. Having a design idea for adaptive software *before* getting down to building it, is a horrifying, if not completely impossible task.

What we need to do is write these programs in such a way that its structure can be modified as and when necessary. The two preferred languages that support this are Apple's Dylan and the Common Lisp Object System (CLOS).

Unlike regular programming, AI programming doesn't use the "if-this-then-that" approach. What it relies on is the encoding of information about the "world" it runs in. When an event occurs in the system, the program uses the information it has about the